

Q-Learning for Mastermind

Rajat Talwar

Department of Computer Science

University of Sheffield

Sheffield, UK

rtalwar2@sheffield.ac.uk

Abstract—This report investigates tabular Q-learning applied to the Mastermind code-breaking game. We implement an epsilon-greedy Q-learning agent and conduct experiments to identify the effect of state representation (history length) and exploration rate (epsilon) on agent performance. Results are compared against a random baseline and analysed in terms of win rate and average turns to solution. All experiments use `NUM_COLORS = 4`, two positions, and a maximum of six turns per episode.

I. INTRODUCTION

Mastermind is a two-player code-breaking game in which one player sets a secret code and the other attempts to guess it within a fixed number of turns. After each guess the guesser receives feedback: *black* pegs for correct colours in the correct position, and *white* pegs for correct colours in the wrong position. In this study we model the single-agent version as a reinforcement learning problem and train a tabular Q-learning agent to solve it.

The agent receives a reward of +10 for guessing the code correctly and -1 for every non-winning step. The state is represented as a tuple of the most recent N turn records, where each record encodes the guess and the resulting feedback. We investigate how the choice of N (history length) and the exploration rate ε affect the agent's ability to learn an effective policy.

II. BACKGROUND

A. Q-Learning

Q-learning is an off-policy temporal-difference algorithm that learns an action-value function $Q(s, a)$, which estimates the expected total discounted reward of taking action a in state s and then following the best possible policy. The update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \eta[y - Q(s, a)], \quad (1)$$

where η is the learning rate and the target y is defined as:

$$y = \begin{cases} r & \text{if terminal} \\ r + \gamma \max_{a'} Q(s', a') & \text{otherwise,} \end{cases} \quad (2)$$

with r the immediate reward, s' the next state, and $\gamma \in [0, 1]$ the discount factor.

B. Epsilon-Greedy Exploration

Action selection follows an ε -greedy policy: with probability ε a uniformly random action is chosen (exploration), and with probability $1 - \varepsilon$ the action with the highest Q-value, $\arg \max_a Q(s, a)$, is chosen (exploitation). During evaluation ε is set to zero so the agent always acts greedily.

C. State Representation

The state passed to the agent is a tuple of the last `history_length` turn records, each of the form (`guess1`, `guess2`, `black`, `white`). Slots for turns not yet played are set to `None`. A larger history gives the agent more information but also increases the state space exponentially, which means more training episodes are needed to learn a good policy.

III. Q1: IMPLEMENTATION

a) (a) The target and terminal vs non-terminal states:

The target y is the value we want $Q(s, a)$ to move towards on each update. In a non-terminal state we do not yet know the true long-run return, so we estimate it using the immediate reward r plus the discounted best value available from the next state, $\gamma \max_{a'} Q(s', a')$. This is called bootstrapping. In a terminal state the episode has ended, so there is no next state and no further reward. The target is therefore just r . If we included the future term at terminal states we would be adding value that does not exist, which would introduce a consistent error into the update and lead the agent to overestimate the value of actions taken just before the episode ends.

b) (b) Role of the learning rate η : The learning rate η controls how much $Q(s, a)$ changes on each update. A high value makes the agent update quickly in response to new experience, but this can cause the values to jump around and not settle. A low value produces more gradual and stable updates, but learning takes longer. A value in between allows the agent to learn reliably without becoming unstable. All experiments in this report fix $\eta = 0.2$.

c) (c) Role of the discount factor γ : The discount factor γ controls how much the agent cares about future rewards compared to immediate ones. A value close to 1 means the agent treats future rewards as nearly as important as immediate ones. A value close to 0 means the agent focuses mainly on the next reward and ignores what happens later. In Mastermind the winning reward of +10 only arrives at the end of the episode, so a high γ is important. It allows the agent to connect its

early guesses to the eventual outcome and learn that working towards the correct code is worthwhile. All experiments fix $\gamma = 0.99$.

IV. EXPERIMENTAL SETUP

All experiments use `NUM_COLORS` = 4, `NUM_POSITIONS` = 2, and `MAX_TURNS` = 6, giving 16 possible secret codes. The random baseline is a uniformly random policy evaluated over 2000 games. Failed games contribute `max_turns` + 1 = 7 to the average-turns metric, and the same penalty is applied to the trained agent. Each experiment runs for 10 000 training episodes per run, with 500 evaluation games used to score each setting. Unless otherwise stated, 10 independent runs are used to compute mean $\pm$ standard deviation. The learning rate and discount factor are fixed at $\eta = 0.2$ and $\gamma = 0.99$ throughout.

V. Q2: HISTORY LENGTH

We compare history lengths $N \in \{1, 2, 3, 4\}$ with $\varepsilon = 0.2$ fixed across 10 independent runs. Table I and Fig. 1 summarise the results.

TABLE I
HISTORY LENGTH COMPARISON (10 RUNS, $\varepsilon = 0.2$)

History	Win Rate (%)	Avg Turns
1	81.9 $\pm$ 13.4	3.92 $\pm$ 0.44
2	100.0 $\pm$ 0.0	3.02 $\pm$ 0.18
3	100.0 $\pm$ 0.0	2.92 $\pm$ 0.16
4	100.0 $\pm$ 0.0	2.89 $\pm$ 0.10
Random baseline	34.3	5.74

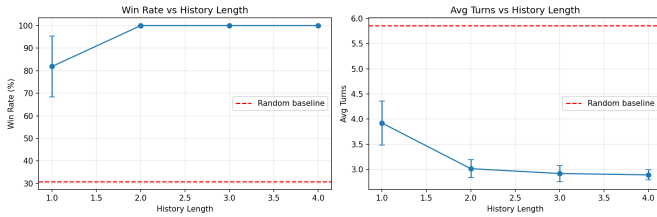


Fig. 1. Win rate and average turns as a function of history length. Error bars show ± 1 standard deviation across 10 runs. The red dashed line is the random baseline.

A. Analysis

a) (a) History length 1: With `history_length` = 1 the agent achieves a mean win rate of 81.9% ($\pm 13.4\%$), which is well above the random baseline of 34.3%, but well below the 100% achieved at all higher history lengths. The large standard deviation shows that performance is inconsistent across runs. This happens because a single turn record is not enough for the state to be Markovian. The agent cannot tell which turn it is on or recall what feedback it received in earlier turns. Without this context, it cannot rule out impossible codes and may repeat guesses or ignore useful information, which limits how well it can perform regardless of how long it trains.

b) (b) Optimal history length: All of $N \in \{2, 3, 4\}$ achieve a 100% win rate, but history length 2 is the most sensible choice. It is the smallest value that achieves perfect win rate, and its average turns (3.02) is only slightly higher than $N = 3$ (2.92) or $N = 4$ (2.89). This result relates to the Markov property: two turns of feedback appear to give the agent enough information to identify which codes are still consistent with all the feedback it has received. This makes the state effectively Markovian. A history of one turn does not satisfy this, while $N \geq 2$ does for this game configuration.

c) (c) State space size and the sample complexity trade-off: Increasing history length causes the state space to grow exponentially. Each turn record can take many different values (all possible guesses combined with all possible feedback), so the number of reachable states grows roughly as $O(k^N)$ for some constant k . With 10 000 training episodes, $N \in \{2, 3, 4\}$ all converge reliably in this configuration, which suggests the state space is still small enough to cover. However, the standard deviation in average turns decreases as N grows, from ± 0.18 at $N = 2$ to ± 0.10 at $N = 4$, which suggests that larger histories do produce slightly more stable policies. In a harder version of the game with more positions or colours, the exponential growth of the state space would likely require many more training episodes to cover properly at high N , making this trade-off much more significant.

VI. Q3: EXPLORATION RATE

Using `history_length` = 2 (chosen in Section V), we sweep $\varepsilon \in \{0.0, 0.1, 0.2, 0.3, 0.5\}$ across 10 independent runs of 10 000 training episodes each, evaluating on 500 games per run. Table II and Fig. 2 summarise the results.

TABLE II
EPSILON SWEEP (10 RUNS, `HISTORY_LENGTH` = 2)

Epsilon	Win Rate (%)	Avg Turns
0.0	100.0 $\pm$ 0.0	3.03 $\pm$ 0.25
0.1	100.0 $\pm$ 0.0	3.16 $\pm$ 0.22
0.2	100.0 $\pm$ 0.0	3.07 $\pm$ 0.18
0.3	100.0 $\pm$ 0.0	2.96 $\pm$ 0.14
0.5	98.5 $\pm$ 4.4	3.06 $\pm$ 0.23
Random baseline	34.8	5.72

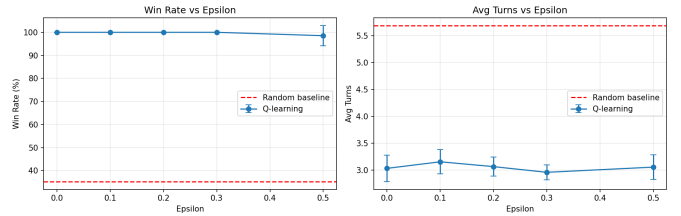


Fig. 2. Win rate and average turns as a function of ε . Error bars show ± 1 standard deviation across 10 runs. The red dashed line is the random baseline.

A. Analysis

a) *(a) Epsilon = 0:* With $\varepsilon = 0$ the agent always selects the action with the highest Q-value during training, yet still achieves a 100% win rate. This is far above the random baseline of 34.8% and matches the best results seen at higher epsilon values. This result can be explained by two features of the setup. First, all unseen state-action pairs start with a Q-value of 0, so at the beginning of training every action in a new state is tied for the highest value. The `get_best_action` method breaks these ties at random, which gives the agent a form of implicit exploration early on before the Q-table has been filled in. Second, the reward structure of -1 per step and $+10$ only for a win means that state-action pairs are quickly assigned different values as training progresses, steering the greedy policy away from losing actions. Because there are only 16 possible codes in this game, this incidental exploration is enough to cover the relevant parts of the state space within 10 000 episodes.

b) *(b) Best epsilon:* $\varepsilon = 0.3$ produces the lowest mean average turns (2.96) and the smallest standard deviation across runs (± 0.14), making it the most efficient and consistent setting tested. All values in $\{0.0, 0.1, 0.2, 0.3\}$ achieve a 100% win rate, so there is no clear winner based on win rate alone. The differences appear mainly in efficiency and consistency. A moderate level of exploration appears to help the agent find slightly shorter solution paths and produce a more reliable policy within the training budget. At $\varepsilon = 0.5$ the win rate drops slightly to 98.5% ($\pm 4.4\%$), which suggests that too much exploration begins to interfere with learning within the available episodes. Based on these results, $\varepsilon = 0.3$ is selected for Experiment 3.

c) *(c) Exploration-exploitation trade-off:* If ε is too low, the agent depends on random tie-breaking for exploration. This means it may not revisit states where Q-values were set early under a poor policy, which can lead to the agent settling on a strategy that is not optimal. If ε is too high, the agent keeps choosing random actions even after it has learned a good policy. This wastes training episodes on actions that do not improve performance, and can reduce the final win rate as seen at $\varepsilon = 0.5$. A value around $\varepsilon = 0.3$ provides a reasonable balance between these two extremes. Evaluation always uses $\varepsilon = 0$ so that the reported results reflect the policy the agent has learned, rather than lucky random guesses.

VII. Q4: LEARNING CURVES

Using `history_length = 2` and $\varepsilon = 0.3$, Fig. 3 shows mean win rate and mean average turns over training episodes, with ± 1 standard deviation bands across 10 independent runs.

A. Analysis

a) *(a) Convergence and plateau:* The agent reached a 100% win rate by the first evaluation checkpoint at 2 000 episodes and maintained this level through to 10 000 episodes, with average turns remaining stable in the range 3.40 to 3.46. Performance improved quickly in the early stages of training and stopped improving well before the training budget ran out.

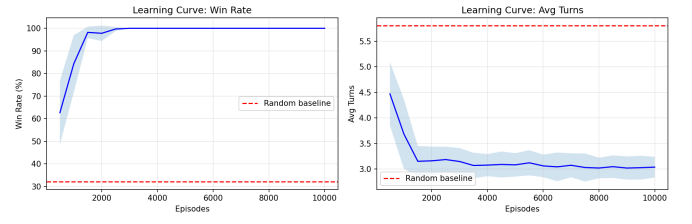


Fig. 3. Learning curves for the final agent configuration (`history_length = 2`, $\varepsilon = 0.3$). Shaded bands show ± 1 standard deviation across 10 runs. The red dashed line is the random baseline.

This suggests the agent had covered the relevant parts of the state space within the first 2 000 episodes. This performance is far above the random baseline of 34.3% win rate and 5.74 average turns, confirming that the agent learned a strong policy early and that additional training beyond 2 000 episodes gave little further benefit.

b) *(b) Multiple runs and stability:* Reporting mean $\pm$ standard deviation across several independent runs is important because RL training involves randomness. Random exploration and random tie-breaking mean a single run could be unusually good or unusually poor, which would give a misleading picture of how the agent typically performs. With 10 runs the win-rate curve showed almost no variation after the first checkpoint, and the average-turns band stayed narrow throughout, which shows that the conclusions were stable at this number of runs. In earlier tests with fewer runs (around 3 to 5), the bands were noticeably wider, suggesting that 10 independent runs is the point where conclusions become reliable for this problem. Running more than 10 runs is unlikely to change the results in any meaningful way.

c) *(c) Average-turns penalty for unsolved games:* Assigning `max_turns + 1 = 7` turns to each failed game means the metric captures both how often the agent wins and how efficiently it does so. If only winning games were counted, an agent that wins rarely but quickly would look efficient even though it fails most of the time. The penalty makes sure that failing to solve the code is properly reflected in the average. This allows a fair comparison between agents that differ in both win rate and speed using a single number. In this experiment the penalty had no visible effect on the learning curve after the first checkpoint, since the agent was already winning all evaluated games by that point.

VIII. CONCLUSION

This study showed that a tabular Q-learning agent can learn to play Mastermind well above the level of a random policy. The main findings were: (i) a history length of at least 2 is needed to give the agent enough information to make good decisions, as a history of 1 achieved only 81.9% win rate with high variability; (ii) history length 2 is the most practical choice since it reaches 100% win rate without unnecessarily growing the state space; (iii) a moderate exploration rate of $\varepsilon = 0.3$ gave the most efficient and stable policy, though $\varepsilon = 0$ also performed well due to implicit exploration from random

tie-breaking in unseen states; and (iv) the agent converged within around 2 000 episodes, well within the 10 000 episode training budget.